CSEg2208 – Design and Analysis of Algorithms

Chapter 5: Divide and Conquer & Decrease and Conquer Algorithms

Outline

- **Introduction**
- **Divide and Conquer Algorithms**
  - **Merge Sort**
  - **Quick Sort**
  - **Binary Search**
  - **Multiplication of Large Numbers**
  - **Strassen's Matrix Multiplication**
- **Decrease and Conquer Algorithms**
  - **Insertion Sort**
  - **Graph Traversal: DFS & BFS**
  - **Topological sort**

Introduction

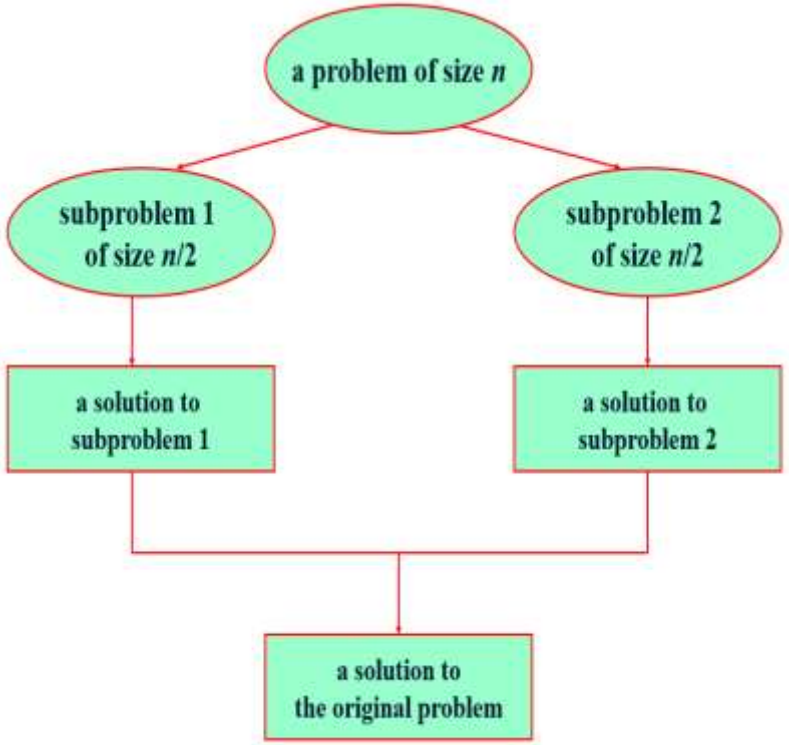
- **Divide-and-conquer** is probably the best-known general algorithm design technique.
  - Divide and Conquer strategy is used if we could break a larger problem into several smaller sub-problems.
  - These sub problems could then be combined if necessary to obtain the solution to the larger problem.
- The **decrease and conquer** technique is similar to divide and conquer, except instead of partitioning a problem into multiple sub-problems of smaller size, we use some technique to reduce our problem into a single problem that is smaller than the original.

Introduction

- **Divide and Conquer VS Decrease and Conquer:**
  - **Decrease and Conquer** involves reducing the problem into 1 sub-problem, while divide and conquer reduces the problem into several sub-problems.
  - That is:- Divide and Conquer would yield something along the lines of $T(\frac{n}{x})$, where x is amount of sub-problems.
  - Whereas Decrease and Conquer would yield something like $T(n - c)$ where c is a constant factor.

Divide and Conquer

- The Divide and Conquer Approach is most-well known algorithm design strategy which follows:
  1. **Divide** instance of problem into two or more smaller instances,
  2. **Solve** smaller instances recursively, and
  3. Obtain solution to original (larger) instance by **combining** these solutions.
- The base case for the recursion are sub-problems of constant size.
- Analysis can be done using **recurrence equations.**



General Divide-and-Conquer Recurrence

- The general Divide-and-Conquer **recurrence** is given as follows:

  $\boldsymbol{T(n) = aT(\frac{n}{b}) + f(n)}$ where $\boldsymbol{f(n) \in \Theta(n^d)}$, $\boldsymbol{d \geq 0}$

  - This recurrence is can be solved using the Master Theorem.

- **Master Theorem**: If $\boldsymbol{f(n) \in \Theta(n^d)}$, $\boldsymbol{d \geq 0}$ in the above recurrence, then:

1. if $f(n)$ is $O(n^{\log_b a-\varepsilon})$, then $T(n)$ is $\Theta(n^{\log_b a})$
2. if $f(n)$ is $\Theta(n^{\log_b a}\log^k n)$, then $T(n)$ is $\Theta(n^{\log_b a}\log^{k+1} n)$
3. if $f(n)$ is $\Omega(n^{\log_b a+\varepsilon})$, then $T(n)$ is $\Theta(f(n))$,

   provided $af(n/b) \leq \delta f(n)$ for some $\delta < 1$.

Or

$$T(n) \in \begin{cases} \Theta(n^d) & \text{if } a < b^d, \\ \Theta(n^d \log n) & \text{if } a = b^d, \\ \Theta(n^{\log_b a}) & \text{if } a > b^d. \end{cases}$$

Divide-and-Conquer Examples

- Example problems best solved using the divide and conquer approach:
  - Sorting: Merge-sort and Quicksort
  - Binary Tree Traversals
  - Multiplication of Large Integers
  - Matrix Multiplication: **Strassen's algorithm**
  - **Closest-pair** and **Convex-hull** algorithms
  - Binary Search: decrease-by-half (or degenerate divide and conquer)

Merge Sort Algorithm

- The **merge sort algorithm** also closely follows the divide-and-conquer paradigm – intuitively, it operates as follows:
  1. **Divide**: Divide the n-element sequence to be sorted into two subsequences of $\frac{n}{2}$ elements each.
  2. **Conquer**: Sort the two subsequences recursively using merge sort.
  3. **Combine**: Merge the two sorted subsequences to produce the sorted answer.

MERGE-SORT $A[1 \,..\, n]$

1. If $n = 1$, done.
2. Recursively sort $A[\,1 \,..\, \lceil n/2 \rceil\,]$ and $A[\,\lceil n/2 \rceil + 1 \,..\, n\,]$.
3. "***Merge***" the 2 sorted lists.

Key subroutine: MERGE

Merge Sort Algorithm

- Split array $A[0 \ldots n-1]$ in two about equal halves and make copies of each half in arrays B and C
  - Sort arrays B and C recursively
  - Merge sorted arrays B and C into array A as follows:
    - Repeat the following until no elements remain in one of the arrays:
      - compare the first elements in the remaining unprocessed portions of the arrays
      - copy the smaller of the two into A, while incrementing the index indicating the unprocessed portion of that array
    - Once all elements in one of the arrays are processed, copy the remaining unprocessed elements from the other array into A.

Merge Sort Algorithm

Pseudocode for Merge Sort and Merge:

ALGORITHM *Mergesort*$(A[0..n-1])$

```
//Sorts array A[0..n − 1] by recursive mergesort
//Input: An array A[0..n − 1] of orderable elements
//Output: Array A[0..n − 1] sorted in nondecreasing order
if n > 1
    copy A[0..⌊n/2⌋ − 1] to B[0..⌊n/2⌋ − 1]
    copy A[⌊n/2⌋..n − 1] to C[0..⌈n/2⌉ − 1]
    Mergesort(B[0..⌊n/2⌋ − 1])
    Mergesort(C[0..⌈n/2⌉ − 1])
    Merge(B, C, A)
```

ALGORITHM *Merge*$(B[0..p-1], C[0..q-1], A[0..p+q-1])$

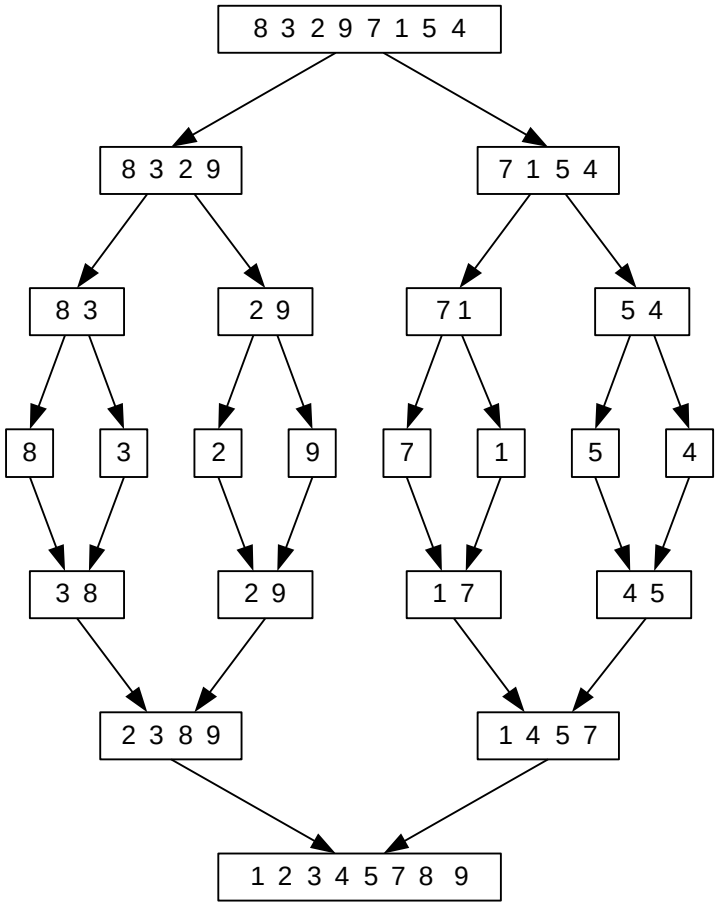
```
//Merges two sorted arrays into one sorted array
//Input: Arrays B[0..p − 1] and C[0..q − 1] both sorted
//Output: Sorted array A[0..p + q − 1] of the elements of B and C
i ← 0; j ← 0; k ← 0
while i < p and j < q do
    if B[i] ≤ C[j]
        A[k] ← B[i]; i ← i + 1
    else A[k] ← C[j]; j ← j + 1
    k ← k + 1
if i = p
    copy C[j..q − 1] to A[k..p + q − 1]
else copy B[i..p − 1] to A[k..p + q − 1]
```

Merge Sort Algorithm

- The key operation of the merge sort algorithm is the merging of two sorted sequences in the "**combine**" step.

- Consider an example of merge sort operation on the array:

$$A = \langle 8, 3, 2, 9, 7, 1, 5, 4 \rangle$$



Analysis of Merge Sort

- Let $\boldsymbol{T(n)}$ be the running time on a problem of size $\boldsymbol{n}$.
  - If the problem size is small enough, say $\boldsymbol{n < c}$ for some constant c, the straightforward solution takes constant time, which we write as $\boldsymbol{\Theta(1)}$.
  - Suppose we divide the problem into sub-problems, each of which is $\frac{1}{b}$ the size of the original.
  - If we take $\boldsymbol{D(n)}$ time to divide the problem into sub problems and $\boldsymbol{C(n)}$ time to combine the solutions to the sub problems into the solution to the original problem, we get the recurrence:

$$T(n) = \begin{cases} \Theta(1) & if\ n < c \\ aT\left(\frac{n}{b}\right) + D(n) + C(n) & otherwise \end{cases}$$

Analysis of Merge Sort

- **Divide**:
  - The divide step just computes the middle of the sub array, which takes constant time, thus, $D(n) = \Theta(1)$.
- **Conquer**:
  - We recursively solve two sub-problems, each of size $\frac{n}{2}$, which contributes $2T(\frac{n}{2})$ to the running time.
- **Combine/Merge**:
  - We have already noted that the **MERGE** procedure on an n-element sub array takes time $\Theta(n)$, so $C(n) = \Theta(n)$.

- Then, the recurrence of merge sort is:

$$T(n) = \begin{cases} \Theta(1) & if\ n = 1 \\ 2T\left(\frac{n}{2}\right) + \Theta(n) & if\ n > 1 \end{cases}$$

- Solving the recurrence, we get $\boldsymbol{T(n) = \Theta(n \log n)}$

Analysis of Merge Sort

- All cases have same efficiency: $\Theta(n \log n)$
  - Number of comparisons in the worst case is close to theoretical minimum for comparison-based sorting:

    - Theoretical min: $\lceil \boldsymbol{log_2 n!} \quad \approx \boldsymbol{n\, log_2 n - 1.44n}$
- **Space requirement:**
  - Book's algorithm: $n + n\, Sum(1 + \frac{1}{2} + \frac{1}{4} + \cdots) = n + n\, \Theta(2) = \Theta(3n)$
  - Can be done $\Theta(2n)$: Don't copy in sort; just specify bounds, copy into extra array on merge, then copy back
    - Any other space required?
  - Can be implemented in place
  - Can be implemented without recursion (bottom-up)

Quicksort Algorithm

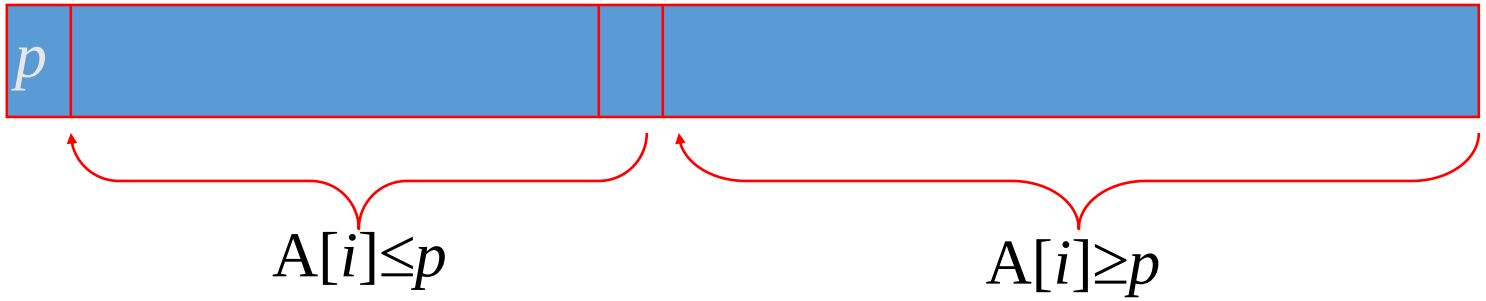
- **Quicksort**, like merge sort, applies the divide-and-conquer method to sort an n element array $A[1:n]$.
  1. **Divide**: Partition the array into two sub-arrays around a ***pivot*** $\boldsymbol{x}$ such that elements in lower sub-array $\leq x \leq$ elements in upper sub-array.

  2. **Conquer**: Recursively sort the two sub-arrays.

  3. **Combine**: Trivial.
     - Key: Linear-time partitioning subroutine.

Quicksort Algorithm

- Select a **pivot** (partitioning element) – here, the first element
  - Rearrange the list so that all the elements in the first $\boldsymbol{s}$ positions are smaller than or equal to the pivot and all the elements in the remaining $n - s$ positions are larger than or equal to the pivot.



  - Exchange the pivot with the last element in the first (i.e., $\leq$) subarray — the pivot is now in its final position
  - Sort the two subarrays recursively.

Quicksort Algorithm

ALGORITHM *Quicksort*$(A[l..r])$

```
//Sorts a subarray by quicksort
//Input: Subarray of array A[0..n − 1], defined by its left and right
//        indices l and r
//Output: Subarray A[l..r] sorted in nondecreasing order
if l < r
    s ← Partition(A[l..r])  //s is a split position
    Quicksort(A[l..s − 1])
    Quicksort(A[s + 1..r])
```

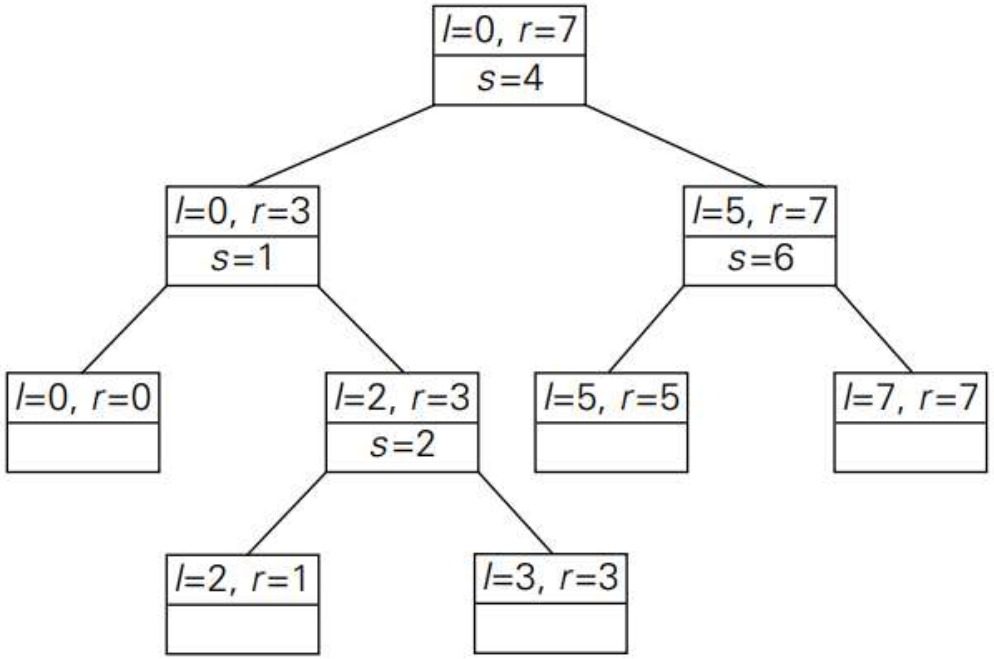
Quicksort Algorithm

ALGORITHM *HoarePartition*$(A[l..r])$

```
//Partitions a subarray by Hoare's algorithm, using the first element
//          as a pivot
//Input: Subarray of array A[0..n − 1], defined by its left and right
//          indices l and r (l < r)
//Output: Partition of A[l..r], with the split position returned as
//          this function's value
p ← A[l]
i ← l; j ← r + 1
repeat
    repeat i ← i + 1 until A[i] ≥ p
    repeat j ← j − 1 until A[j] ≤ p
    swap(A[i], A[j])
until i ≥ j
swap(A[i], A[j])   //undo last swap when i ≥ j
swap(A[l], A[j])
return j
```

Quicksort Algorithm

- Example of quicksort operation:
  - On $A = \langle 5, 3, 1, 9, 8, 2, 4, 7 \rangle$



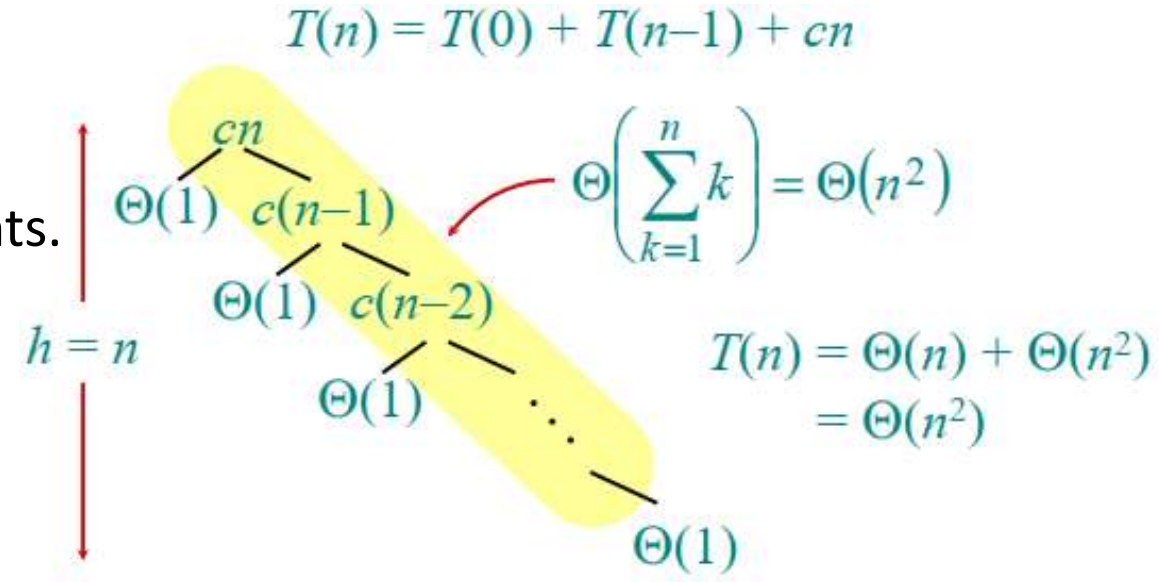
Recursion Tree

0	1	2	3	4	5	6	7
5	*i* 3	1	9	8	2	4	*j* 7
5	3	1	*i* 9	8	2	*j* 4	7
5	3	1	*i* 4	8	2	*j* 9	7
5	3	1	4	*i* 8	*j* 2	9	7
5	3	1	4	*i* 2	*j* 8	9	7
5	3	1	4	*j* 2	*i* 8	9	7
2	3	1	4	**5**	8	9	7
2	*i* 3	1	*j* 4				
2	*i* 3	*j* 1	4				
2	*i* 1	*j* 3	4				
2	*j* 1	*i* 3	4				
1	**2**	3	4				
1							
		3	*i j* 4				
		j **3**	*i* 4				
			4				
					8	*i* 9	*j* 7
					8	*i* 7	*j* 9
					8	*j* 7	*i* 9
					7	**8**	9
					7		
							9

Analysis of Quicksort Algorithm

- Assume all input elements are distinct.
- In practice, there are better partitioning algorithms for when duplicate input elements may exist.
- Let $T(n) =$ worst-case running time on an array of n elements.

- **Worst-case of partitioning**
  - Input sorted or reverse sorted.
  - Partition around min or max element.
  - One side of partition always has no elements.



Analysis of Quicksort Algorithm

Recurrences:

- Best case: $T(n) = T(\frac{n}{2}) + \Theta(n)$ – if balanced partitioning
- Worst case: $T(n) = T(n-1) + \Theta(n)$ – if list is sorted or reversely sorted.

Performance:

- Best case: split in the middle — $\Theta(n \log n)$
- Worst case: sorted array! — $\Theta(n^2)$
- Average case: random arrays — $\Theta(n \log n)$

Analysis of Quicksort Algorithm

- **Problems**:
  - Duplicate elements

- **Improvements**:
  - better pivot selection: median of three partitioning
  - switch to insertion sort on small sub-arrays
  - elimination of recursion

  These combine to 20-25% improvement
  - Dual Pivot

- Method of choice for internal sorting of large files ($n \geq 10000$)

Binary Search Algorithm

- **Binary Search** is an extremely well-known instance of divide-and-conquer paradigm.
  - Given an ordered array of $\boldsymbol{n}$ elements, the basic idea of binary search is that for a given element we "probe" the middle element of the array.
  - We continue in either the lower or upper segment of the array, depending on the outcome of the probe until we reached the required (given) element.

Algorithm: $BinarySearch(A, x)$

Input: Sorted array A of size n, an element x to be searched

Question: Is $x \in A$

Approach: Check whether $A[\frac{n}{2}] = x$.

If $x > A[\frac{n}{2}]$, then prune the lower half of the array, $A[1, \ldots, \frac{n}{2}]$.

Otherwise, prune the upper half of the array.

Analysis of Binary Search Algorithm

- **Recurrence**:

$$T(n) = T(\frac{n}{2^k}) \ + \ 1 \ + \cdots \ + 1$$

  - Since $T(1) = 1$, when $n = 2^k$, $T(n) = T(1) + k = 1 + \log_2(n)$.

$$\log_2(n) \ \leq \ 1 \ + \ \log_2(n) \ \leq \ 2\log_2(n), \forall\, n \ \geq \ 2$$

  - Therefore, $T(n) \ = \ \Theta(\log_2 n)$.

Multiplication of Large Integers

- Consider the problem of multiplying two (large) *n*-digit integers represented by arrays of their digits such as:

$$A = 12345678901357986429 \quad B = 87654321284820912836$$

  - The grade-school algorithm:

$$\begin{array}{r}
a_1\ a_2 \ldots\ a_n \\
b_1\ b_2 \ldots\ b_n \\
\hline
(d_{10})\ d_{11}d_{12} \ldots d_{1n} \\
(d_{20})\ d_{21}d_{22} \ldots d_{2n} \\
\ldots\ \ldots\ \ldots\ \ldots\ \ldots\ \ldots\ \ldots \\
(d_{n0})\ d_{n1}d_{n2} \ldots d_{nn}
\end{array}$$

  - Efficiency: $M(n) = n^2$ - one-digit multiplications.

Multiplication of Large Integers

First Divide-and-Conquer Algorithm:

- A small example: $A \cdot B$ where $A = 2135$ and $B = 4014$

$$A = (21 \cdot 10^2 + 35), \qquad B = (40 \cdot 10^2 + 14)$$

  - So, $A \cdot B = (21 \cdot 10^2 + 35) * (40 \cdot 10^2 + 14)$

$$= 21 * 40 \cdot 10^4 + (21 * 14 + 35 * 40) \cdot 10^2 + 35 * 14$$

- In general, if $A = A_1 A_2$ and $B = B_1 B_2$ (where A and B are n-digit, and A_1, A_2, B_1, B_2 are $\frac{n}{2}$ −digit numbers),

$$A * B = A_1 * B_1 \cdot 10^n + (A_1 * B_2 + A_2 * B_1) \cdot 10^{\frac{n}{2}} + A_2 * B_2$$

- Recurrence for the number of one-digit multiplications $M(n)$:

$$M(n) = 4M(\frac{n}{2}), \qquad M(1) = 1$$

- Solution: $M(n) = n^2$

Multiplication of Large Integers

Second Divide-and-Conquer Algorithm:

$$A \quad B = A_1 * B_1 \cdot 10^n + (A_1 * B_2 + A_2 * B_1) \cdot 10^{\frac{n}{2}} + A_2 * B_2$$

- The idea is to decrease the number of multiplications from 4 to 3:

$$(A_1 + A_2)*(B_1 + B_2) = A_1 * B_1 + (A_1 * B_2 + A_2 * B_1) + A_2 * B_2$$

  - I.e.,

$$(A_1*B_2 + A_2*B_1) = (A_1 + A_2)*(B_1 + B_2) - A_1*B_1 - A_2*B_2$$

  - Which requires only 3 multiplications at the expense of (4-1=3) extra additions and subtractions (2 adds and 2 subs – 1 add)

- **Recurrence** for the number of multiplications $M(n)$:

$$M(n) = 3M(\frac{n}{2}), \qquad M(1) = 1$$

- **Solution**: $M(n) = 3^{\log_2 n} = n^{\log_2 3} \approx n^{1.585}$

Multiplication of Large Integers

Second Divide-and-Conquer Algorithm:

- **Exercise**: Multiply $2135 * 4014$ using D&C?

Strassen's Matrix Multiplication

- **Strassen** observed [1969] that the product of two matrices can be computed as follows:

$$\begin{bmatrix} C_{00} & C_{01} \\ C_{10} & C_{11} \end{bmatrix} = \begin{bmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{bmatrix} * \begin{bmatrix} B_{00} & B_{01} \\ B_{10} & B_{11} \end{bmatrix}$$

$$= \begin{bmatrix} M_1 + M_4 - M_5 + M_7 & M_3 + M_5 \\ M_2 + M_4 & M_1 + M_3 - M_2 + M_6 \end{bmatrix}$$

- **Example**: $M_2 + M_4 = (A_{10} + A_{11}) * B_{00} + A_{11} * (B_{10} - B_{00})$

$$= A_{10}\, B_{00} + A_{11} B_{10}$$

Strassen's Matrix Multiplication

- Formulas for Strassen's Algorithm

$$M_1 = (A_{00} + A_{11}) * (B_{00} + B_{11})$$

$$M_2 = (A_{10} + A_{11}) * B_{00}$$

$$M_3 = A_{00} * (B_{01} + B_{11})$$

$$M_4 = A_{11} * (B_{10} + B_{00})$$

$$M_5 = (A_{00} + A_{01}) * B_{11}$$

$$M_6 = (A_{10} + A_{00}) * (B_{00} + B_{01})$$

$$M_7 = (A_{01} + A_{11}) * (B_{10} + B_{11})$$

Analysis of Strassen's Algorithm

- If n is not a power of 2, matrices can be padded with zeros.
- Number of multiplications:

$$M(n) = 7M(\frac{n}{2}), \quad M(1) = 1$$

  - **Solution**: $M(n) = 7^{\log 2n} = n^{log\ 2^7} \approx n^{2.807}$ vs. n^3 of brute-force alg.

- Algorithms with better asymptotic efficiency are known but they are even more complex.

Strassen's Matrix Multiplication

- **Exercise**: Apply Strassen's Matrix Multiplication Algorithm to multiply matrices below:

$$\begin{bmatrix} 1 & 0 & 2 & 1 \\ 4 & 1 & 1 & 0 \\ 0 & 1 & 3 & 0 \\ 5 & 0 & 2 & 1 \end{bmatrix} * \begin{bmatrix} 0 & 1 & 0 & 1 \\ 2 & 1 & 0 & 4 \\ 2 & 0 & 1 & 1 \\ 1 & 3 & 5 & 0 \end{bmatrix}$$

Decrease and Conquer Algorithms

- The **decrease-and-conquer** technique is based on exploiting the relationship between a solution to a given instance of a problem and a solution to its smaller instance.
  - Once such a relationship is established, it can be exploited either **top down** or **bottom up**.
  - The top-down approach leads naturally to a recursive implementation, although, an ultimate implementation may well be non-recursive.
  - The bottom-up variation is usually implemented iteratively, starting with a solution to the smallest instance of the problem; it is called sometimes the ***incremental approach***.

Decrease and Conquer Algorithms

- The decrease-and-conquer approach works as follows:
  1. Reduce problem instance to smaller instance of the same problem
  2. Solve smaller instance
  3. Extend solution of smaller instance to obtain solution to original instance
- Can be implemented either top-down (inductive) or bottom-up (incremental approach).
- There are three major variations of decrease-and-conquer:
  1. **Decrease by a constant**
  2. **Decrease by a constant factor**
  3. **Decrease by a variable size**

Decrease and Conquer Algorithms

Three Major Variations

1. **Decrease by a constant (usually by 1):**
   - Insertion sort
   - Graph traversal algorithms (DFS and BFS)
   - Topological sorting
   - Algorithms for generating permutations, subsets
2. **Decrease by a constant factor (usually by half)**
   - Binary search and bisection method
   - Exponentiation by squaring
   - Multiplication à la russe
3. **Variable-size decrease**
   - Euclid's algorithm
   - Selection by partition
   - Nim-like games
   - This usually results in a recursive algorithm.

Decrease and Conquer Algorithms

What is the difference?

- Consider the problem of exponentiation: Compute x^n

  - **Brute Force**: $n - 1$ multiplications
  - **Divide and conquer**: $T(n) = 2 * T\left(\frac{n}{2}\right) + 1 = n - 1$
  - **Decrease by one**: $T(n) = T(n-1) + 1 = n - 1$
  - **Decrease by constant factor**: $T(n) = T\left(\frac{n}{a}\right) + a - 1 = (a-1)\log_a n = \log_2 n$
    where $a = 2$

Insertion Sort

- To sort array $\boldsymbol{A[0..n-1]}$, sort $\boldsymbol{A[0..n-2]}$ recursively and then insert $\boldsymbol{A[n-1]}$ in its proper place among the sorted $\boldsymbol{A[0..n-2]}$.
  - Usually implemented bottom-up (non-recursively)
- **Example**: **Sort 6, 4, 1, 8, 5** using the insertion sort algorithm:

$$
\begin{array}{l}
6 \mid 4 \quad 1 \quad 8 \quad 5 \\
4 \quad 6 \mid 1 \quad 8 \quad 5 \\
1 \quad 4 \quad 6 \mid 8 \quad 5 \\
1 \quad 4 \quad 6 \quad 8 \mid 5 \\
1 \quad 4 \quad 5 \quad 6 \quad 8
\end{array}
$$

Insertion Sort

ALGORITHM *InsertionSort*$(A[0..n-1])$

```
//Sorts a given array by insertion sort
//Input: An array A[0..n − 1] of n orderable elements
//Output: Array A[0..n − 1] sorted in nondecreasing order
for i ← 1 to n − 1 do
    v ← A[i]
    j ← i − 1
    while j ≥ 0 and A[j] > v do
        A[j + 1] ← A[j]
        j ← j − 1
    A[j + 1] ← v
```

Analysis of Insertion Sort

- **Time efficiency:**
  $C_{worst}(n) = \frac{n(n-1)}{2} \in \Theta(n^2)$
  $C_{avg}(n) \approx \frac{n^2}{4} \in \Theta(n^2)$
  $C_{avg}(n) = n - 1 \in \Theta(n)$ (also fast on almost sorted arrays)
- **Space efficiency**: in-place
- **Stability**: yes
  - Best elementary sorting algorithm overall
  - Binary insertion sort

Graph Traversal: DFS & BFS

- Many problems require processing all graph vertices (and edges) in systematic fashion.

- There are two graph traversal algorithms:

  1. Depth-first search (DFS)

  2. Breadth-first search (BFS)

Graph Traversal: DFS & BFS

Depth–First–Search (DFS):

- Visits graph's vertices by always moving away from last visited vertex to an unvisited one, backtracks if no adjacent unvisited vertex is available.

- Recursive or it uses a stack
  - a vertex is pushed onto the stack when it's reached for the first time
  - a vertex is popped off the stack when it becomes a dead end, i.e., when there is no adjacent unvisited vertex

- "**Redraws**" graph in tree-like fashion (with tree edges and back edges for undirected graph)

Depth–First–Search (DFS)

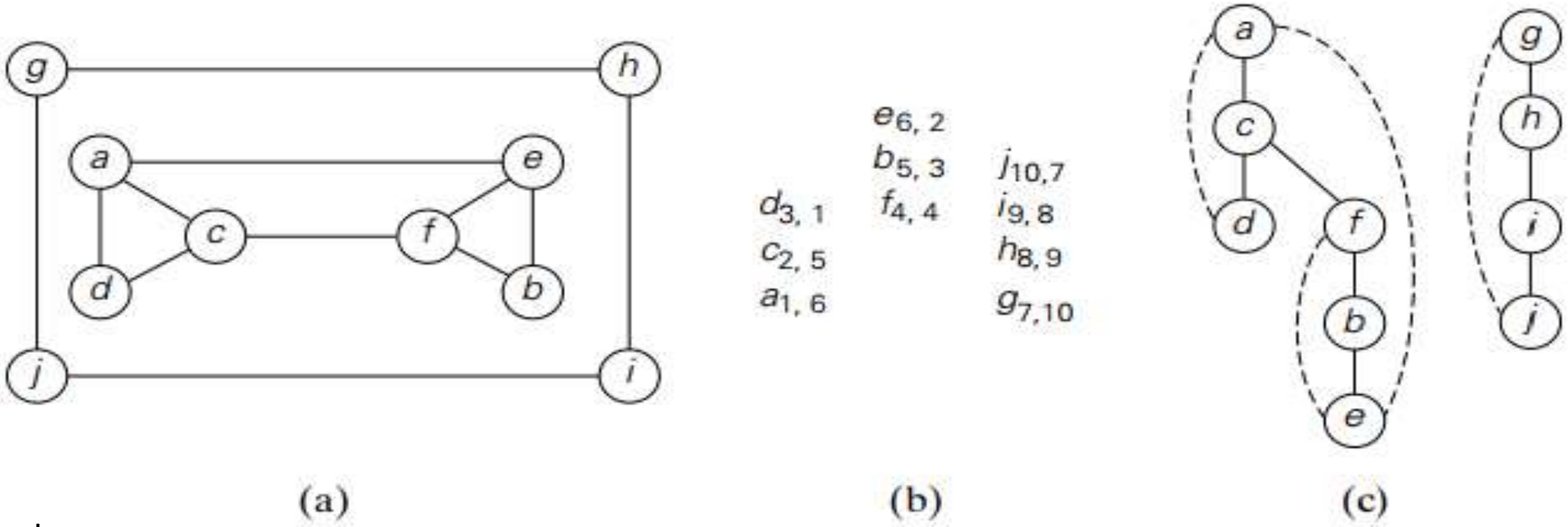
ALGORITHM *DFS(G)*

```
//Implements a depth-first search traversal of a given graph
//Input: Graph G = ⟨V, E⟩
//Output: Graph G with its vertices marked with consecutive integers
//        in the order they are first encountered by the DFS traversal
mark each vertex in V with 0 as a mark of being "unvisited"
count ← 0
for each vertex v in V do
    if v is marked with 0
        dfs(v)

dfs(v)
//visits recursively all the unvisited vertices connected to vertex v
//by a path and numbers them in the order they are encountered
//via global variable count
count ← count + 1;   mark v with count
for each vertex w in V adjacent to v do
    if w is marked with 0
        dfs(w)
```

Depth–First–Search (DFS)

- **Example**: DFS traversal of undirected graph



(a) graph,
(b) traversal stack, first subscript = order vertex visited (pushed on stack), second subscript = order of being dead-end (popped off the stack).
(c) DFS forest with the tree and back edges shown with solid and dashed lines, respectively.

Depth–First–Search (DFS)

- Notes on DFS:
  - **DFS can be implemented with graphs represented as:**
    - adjacency matrices: $\Theta(|V|^2)$. Why?
    - adjacency lists: $\Theta(|V| + |E|)$. Why?

  - **Yields two distinct ordering of vertices:**
    - order in which vertices are first encountered (pushed onto stack)
    - order in which vertices become dead-ends (popped off stack)

  - **Applications:**
    - checking connectivity, finding connected components
    - checking acyclicity (if no back edges)
    - finding articulation points and biconnected components
    - searching the **state-space** of problems for solutions (in AI)

Breadth–First–Search (BFS)

- Visits graph vertices by moving across to all the neighbors of the last visited vertex.
  - Instead of a stack, BFS uses a **queue**
  - Similar to level-by-level tree traversal
  - "**Redraws**" graph in tree-like fashion (with tree edges and cross edges for undirected graph)

Breadth–First–Search (BFS)

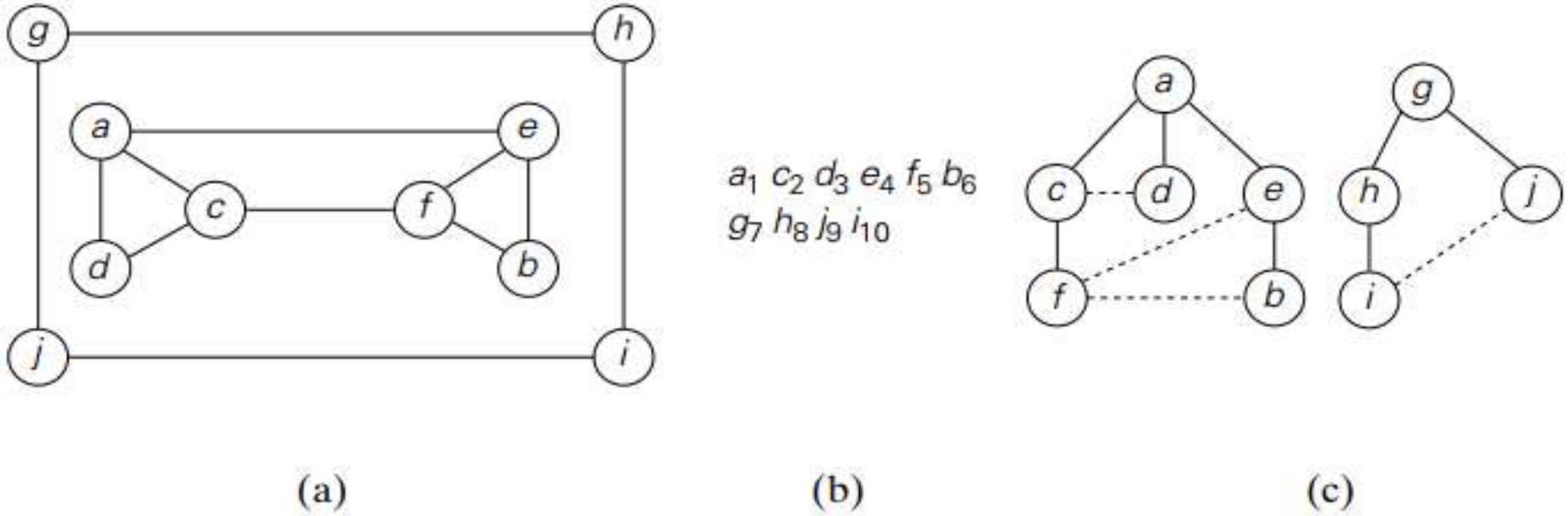
ALGORITHM *BFS*(G)

```
//Implements a breadth-first search traversal of a given graph
//Input: Graph G = ⟨V, E⟩
//Output: Graph G with its vertices marked with consecutive integers
//        in the order they are visited by the BFS traversal
mark each vertex in V with 0 as a mark of being "unvisited"
count ← 0
for each vertex v in V do
    if v is marked with 0
        bfs(v)

bfs(v)
//visits all the unvisited vertices connected to vertex v
//by a path and numbers them in the order they are visited
//via global variable count
count ← count + 1;   mark v with count and initialize a queue with v
while the queue is not empty do
    for each vertex w in V adjacent to the front vertex do
        if w is marked with 0
            count ← count + 1;   mark w with count
            add w to the queue
    remove the front vertex from the queue
```

Breadth–First–Search (BFS)

- **Example**: BFS traversal of undirected graph



(a) Graph
(b) Traversal queue, with the numbers indicating the order in which the vertices are visited, i.e., added to (and removed from) the queue.
(c) BFS forest with the tree and cross edges shown with solid and dotted lines, respectively.

Breadth–First–Search (BFS)

- Notes on BFS:
  - BFS has same efficiency as DFS and can be implemented with graphs represented as:
    - adjacency matrices: $\Theta(|V|^2)$. Why?
    - adjacency lists: $\Theta(|V| + |E|)$. Why?

  - Yields single ordering of vertices (order added/deleted from queue is the same).

  - **Applications**: same as DFS, but can also find paths from a vertex to all other vertices with the smallest number of edges.

Main Facts about DFS and BFS

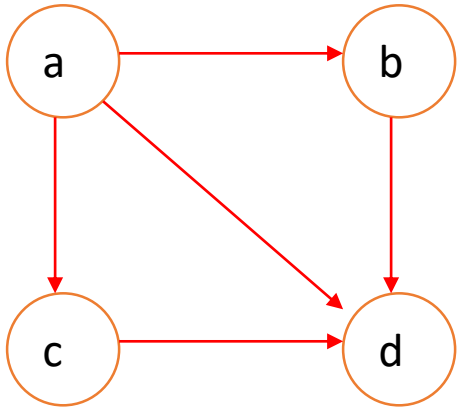
	DFS	**BFS**
Data structure	a stack	a queue
Number of vertex orderings	two orderings	one ordering
Edge types (undirected graphs)	tree and back edges	tree and cross edges
Applications	connectivity, acyclicity, articulation points	connectivity, acyclicity, minimum-edge paths
Efficiency for adjacency matrix	$\Theta(\lvert V^2 \rvert)$	$\Theta(\lvert V^2 \rvert)$
Efficiency for adjacency lists	$\Theta(\lvert V \rvert + \lvert E \rvert)$	$\Theta(\lvert V \rvert + \lvert E \rvert)$

Topological Sort

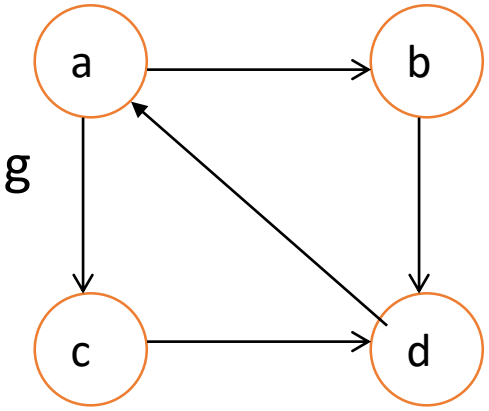
DAGs and Topological Sorting

- A **dag**: a directed acyclic graph, i.e. a directed graph with no (directed) cycles.

a dag



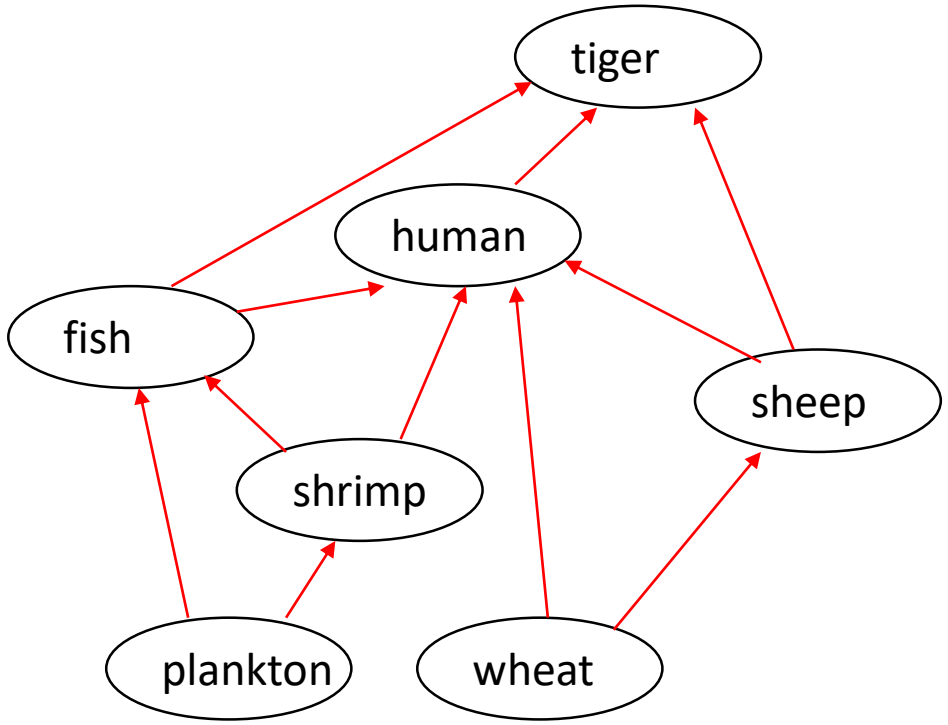
not a dag



  - Arise in modeling many problems that involve prerequisite constraints (construction projects, document version control)
  - Vertices of a dag can be linearly ordered so that for every edge its starting vertex is listed before its ending vertex (**topological sorting**).
  - Being a dag is also a necessary condition for topological sorting to be possible.

Topological Sort

- **Topological Sorting Example**: Order the following items in the food chain: tiger, human, wheat, shrimp, fish, sheep, and plankton.

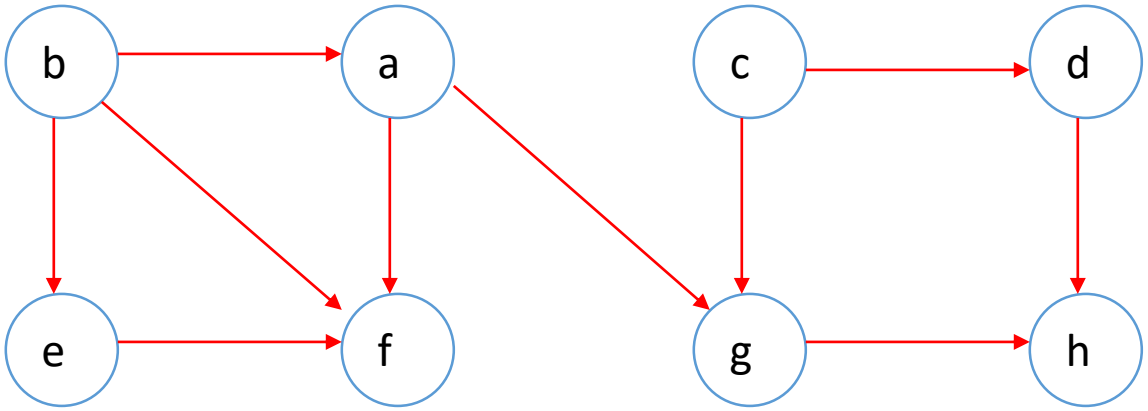


Topological Sort

<u>DFS-based Algorithm for Topological Sorting:</u>

- Perform DFS traversal, noting the order vertices are popped off the traversal stack
- Reverse order solves topological sorting problem
- Back edges encountered?→ NOT a dag!

- **Example**:

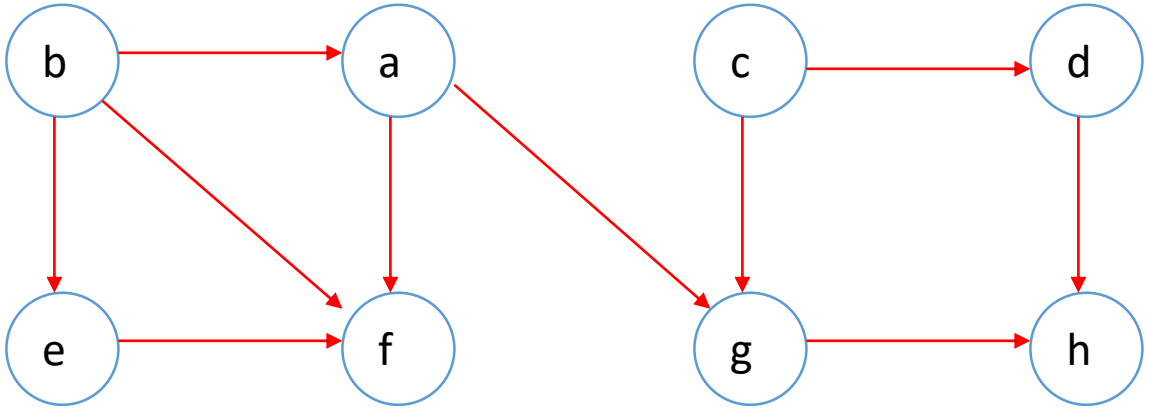


- **Efficiency**: The same as that of DFS.

Topological Sort

<u>Source Removal Algorithm:</u>

- Repeatedly identify and remove a source (a vertex with no incoming edges) and all the edges incident to it until either no vertex is left or there is no source among the remaining vertices (not a dag).
- **Example**:



- **Efficiency**: same as efficiency of the DFS-based algorithm, but how would you identify a source? How do you remove a source from the dag?
  - "Invert" the adjacency lists for each vertex to count the number of incoming edges by going through each adjacency list and counting the number of times that each vertex appears in these lists.
  - To remove a source, decrement the count of each of its neighbors by one.

End of CH5!!!

- **Thank You!!!**